Returning a JSON response : Code-Along (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-json-response)  (https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-json-response/issues/new)

Learning Goals

- Use a dictionary to store row data from a query.
- Implement a Flask application that returns `JSON` in the response.

Key Vocab

- **JSON**: JavaScript Object Notation, is a *data interchange format* often used for transmitting data between a client and a server.
- **Dictionary**: A data structure used to store data values in key:value pairs.

Introduction

We've seen how to generate a server response as a string containing `HTML`. However, web servers often generate responses in different text formats such as `JSON` or `XML`. For example, a server might pass data formatted as a JSON string to a React front-end, which might create various components based on the JSON data.

In this lesson, we'll adapt our Flask application to return `JSON` in the response for each view.

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

The commands `flask db init` and `flask db migrate` have already been run. Run the following command to initialize the database from the existing migration script:

```
$ flask db upgrade head
```

Run the following command to seed the table with sample data:

```
$ python seed.py
```

Use the Flask shell to confirm 10 random random pets have been added to the database (your results will differ):

```
$ flask shell
>>> Pet.query.all()
[<Pet 1, Robin, Hamster>, <Pet 2, Gwendolyn, Dog>, <Pet 3, Michael, Turtle>, <Pet 4, Austin, Cat>, <Pet 5, Jennifer,
>>>
```

Review: What is JSON Again?

JSON, JavaScript Object Notation, is a *data interchange format* often used for transmitting data between a client and a server. JSON data is stored in a Python application as a `String`, but structured in a way that looks very similar to a JavaScript object.

Below is an example of what a piece of JSON looks like when sent from client to server (or vice versa):

```
'{ "name": "Annie Easley", "occupation": "Computer Scientist" }'
```

The data above is a `String`, but you can see that it contains what looks like key/value pairs. The keys and values are both wrapped in quotes while other characters, `{`, `}`, and `:`, are not. This is required syntax for JSON. All text-based data, even keys, must be wrapped in quotes within the larger `String`. Numbers are the only exception to this.

When working with JSON, the outside quotes are not always shown. Instead of a single line like above, we may see JSON like this:

```
{
  "name": "Annie Easley",
  "occupation": "Computer Scientist"
}
```

Returning a JSON response from a Flask view

Take a look at `server/app.py`. The `index()` view returns a string containing HTML, while the `demo_json()` view returns a JSON formatted string.

```
# server/app.py
#!/usr/bin/env python3

from flask import Flask, make_response
from flask_migrate import Migrate

from models import db, Pet

app = Flask(__name__)
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

migrate = Migrate(app, db)
db.init_app(app)

@app.route('/')
def index():
    return make_response(
        '<h1>Welcome to the pet directory!</h1>',
        200
    )

@app.route('/demo_json')
def demo_json():
    pet_json = '{"id": 1, "name" : "Fido", "species" : "Dog"}'
    return make_response(pet_json, 200)

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

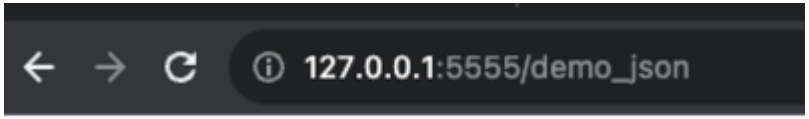
Look carefully at the structure of the Python string assigned to the variable `json` within the `demo_json()` view. The Python string itself is enclosed in single quotes, while the keys and values (except for the number) are enclosed in double quotes.

Let's test this view. Make sure you are in the `server` directory, then run `app.py`:

```
$ python app.py
```

You can also run `flask run` to start the server.

Open a browser window and navigate to **http://127.0.0.1:5555/demo_json (http://127.0.0.1:5555/demo_json)**. The browser displays the JSON data:



{"id": 1, "name" : "Fido", "species" : "Dog"}

Notice the response does not include the surrounding single quotes. While the Python program needs quotes to define a string literal, the web browser displays JSON without the outer quotes.

Our ultimate goal is to query the database and substitute the pet's id, name, and species into the string. The easiest way to do this is to create a dictionary to hold a pet's data. Recall that a dictionary is used to store data values in key:value pairs.

Update `app.py` to delete this statement:

```
pet_json = '{"id": 1, "name" : "Fido", "species" : "Dog"}'
```

Replace it with a statement that uses a dictionary to map pet attributes to values:

```
pet_dict = {'id': 1,
            'name': 'Fido',
            'species': 'Dog'
            }
```

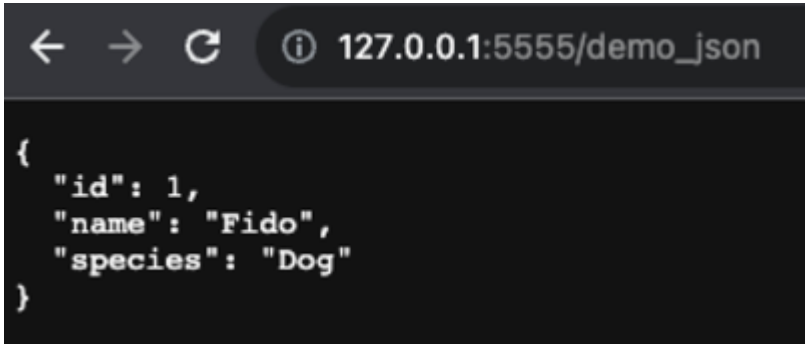
The dictionary looks very similar to the JSON format except (1) there are no outer single quotes since it is not a string, and (2) single quotes are often used for keys as they are string literals. Single or double quotes can be used for the value.

The view should now look like this:

```
@app.route('/demo_json')
def demo_json():
    pet_dict = {'id': 1,
                'name': 'Fido',
                'species': 'Dog'
```

```
                }
    return make_response(pet_dict, 200)
```

Let's refresh the web page in the browser.



Notice the browser displays JSON (double quotes) even though the dictionary uses single quotes.

When a dictionary is passed into `make_response()`, Flask automatically transforms the dictionary into JSON by calling a method named `jsonify()`. You might occasionally see code containing explicit calls to `jsonify()`, since older versions of Flask did not automatically call it on dictionaries.

You may see the JSON displayed on a single line if you started the app with `flask run`. To display each key/value pair on a separate line, update `app.py` to add a configuration setting `app.json.compact = False` as shown below: `

```
app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False
```

Now let's evolve the view to get the first pet from the `pets` table and use the query result for the dictionary values.

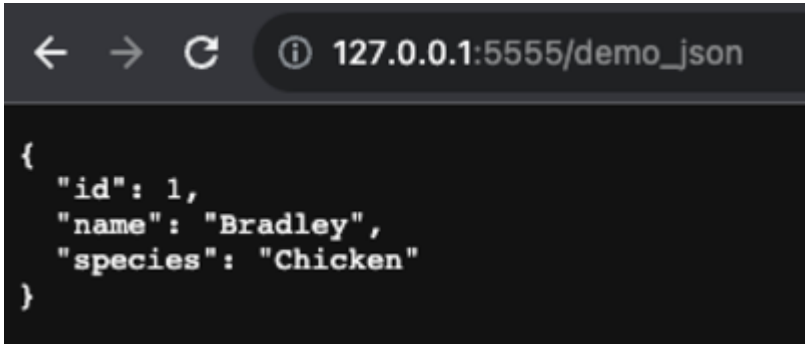
Update the `demo_json()` view as shown:

```
@app.route('/demo_json')
def demo_json():
```

```
    pet = Pet.query.first()
    pet_dict = {'id': pet.id,
                'name': pet.name,
                'species': pet.species
                }

    return make_response(pet_dict, 200)
```

Refresh the web page. Assuming you seeded the database table, you should see a random pet name and species:



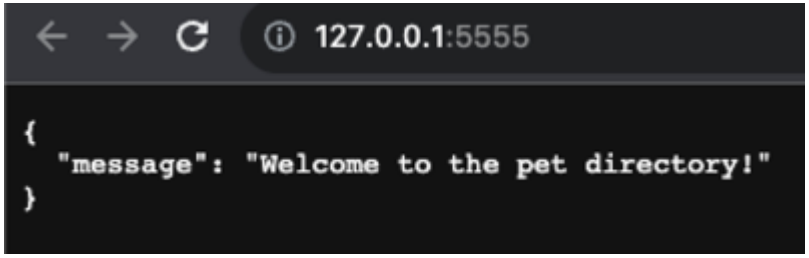
So now we know how to query a database, create a dictionary from the query result, and pass the dictionary to `make_response()` to produce a JSON response.

Let's evolve each route from the previous lesson to produce a JSON response. Keep in mind we might eventually create a React front-end client that would create components using the JSON data.

Update the `index()` view as shown. The response body will consist of a key named `message` and a value `'Welcome to the pet directory!'` .

```
@app.route('/')
def index():
    body = {'message': 'Welcome to the pet directory!'}
    return make_response(body, 200)
```

Let's test the route with **http://127.0.0.1:5555/ (http://127.0.0.1:5555/)** .



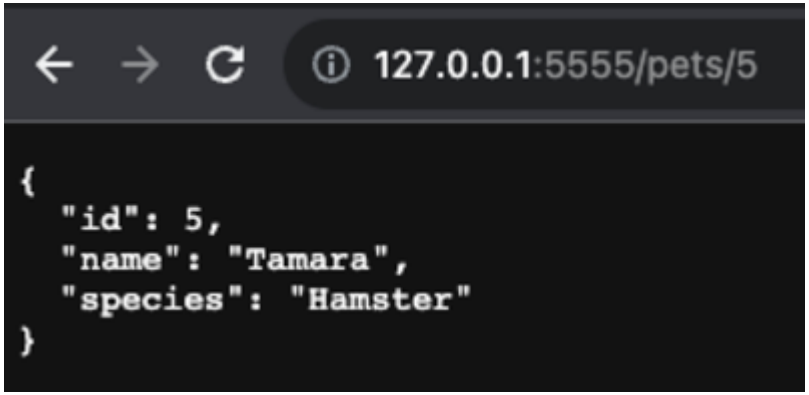
Let's add a view to get pet data for a given id value. The code is similar to the example `demo_json()` view, except the route takes the id as a parameter. We'll also include an error response if the pet is not found.

```
@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        body = {'id': pet.id,
                'name': pet.name,
                'species': pet.species}
        status = 200
    else:
        body = {'message': f'Pet {id} not found.'}
        status = 404

    return make_response(body, status)
```

Test the new view with a valid id using the URL **http://127.0.0.1:5555/pets/5 (http://127.0.0.1:5555/pets/5)** .



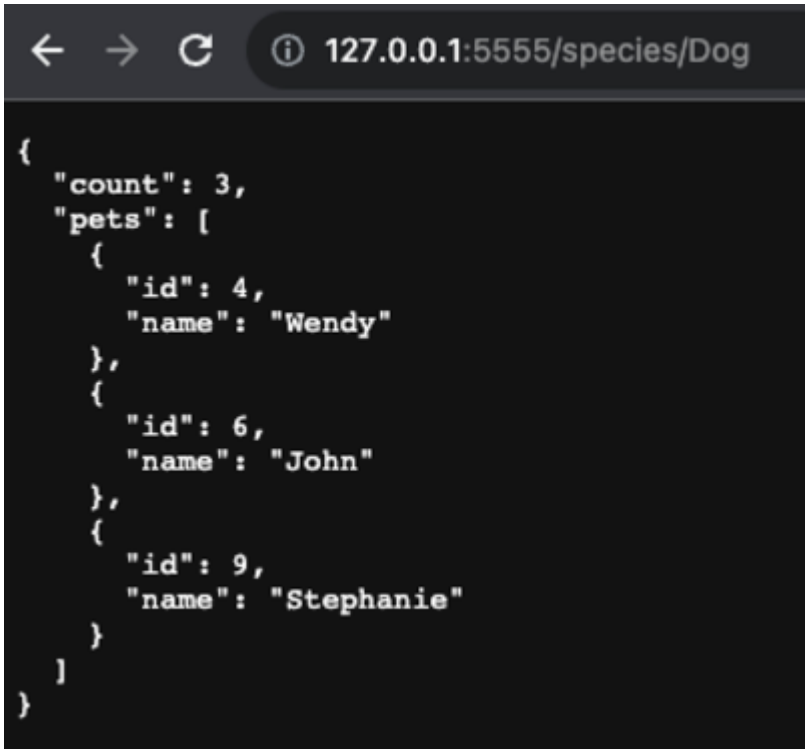
Test the view with a non-existent id such the URL **http://127.0.0.1:5555/pets/1000 (http://127.0.0.1:5555/pets/1000)** .

Finally, let's add a view to get pets based on species. Since there might be several pets for a given species, we need to use a loop to create a dictionary for each pet returned from the query. The array containing all of the pet dictionaries is then used as the value in the dictionary entry.

```
@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = []  # array to store a dictionary for each pet
    for pet in Pet.query.filter_by(species=species).all():
        pet_dict = {'id': pet.id,
                    'name': pet.name,
                    }
        pets.append(pet_dict)
    body = {'count': len(pets),
            'pets': pets
```

```
            }
    return make_response(body, 200)
```

Test the new view with various values for species such as Dog using the URL [http://127.0.0.1:5555/species/Dog (http://127.0.0.1:5555/species/Dog)](http://127.0.0.1:5555/species/Dog).



Conclusion

A Flask application can return a result in many formats, include JSON. A dictionary is a key/value data structure that looks similar to JSON, yet it is not a string. Flask automatically transforms a dictionary to JSON if one is passed to the `make_response()` function. By returning JSON instead of HTML, a client application can transform the server response into a variety of user interface components.

Solution

We can delete the `demo_json()` view since `pet_by_id()` covers similar functionality but with a route parameter and error handling.

```
# server/app.py
#!/usr/bin/env python3

from flask import Flask, make_response
from flask_migrate import Migrate

from models import db, Pet

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)
db.init_app(app)

@app.route('/')
def index():
    body = {'message': 'Welcome to the pet directory!'}
    return make_response(body, 200)

@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        body = {'id': pet.id,
                'name': pet.name,
                'species': pet.species}
        status = 200
    else:
```

```
        body = {'message': f'Pet {id} not found.'}
        status = 404

    return make_response(body, status)

@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = []  # array to store a dictionary for each pet
    for pet in Pet.query.filter_by(species=species).all():
        pet_dict = {'id': pet.id,
                    'name': pet.name,
                    }
        pets.append(pet_dict)
    body = {'count': len(pets),
            'pets': pets
            }
    return make_response(body, 200)

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- **Python Dictionary (https://docs.python.org/3/tutorial/datastructures.html#dictionaries)**
- **JSON (https://www.json.org/json-en.html)**

This tool needs to be loaded in a new browser window

Load Returning a JSON response : Code-Along (CodeGrade) in a new window